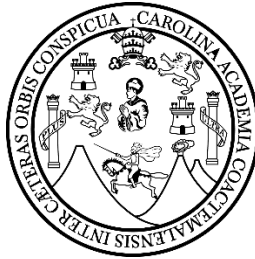


UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



FACULTAD DE INGENIERIA

GESTION DE BANCOS Y FACTURAS

DOCUMENTACIÓN

SERGIO RAÚL QUEZEL YOL
ESTUDIANTE DE CIENCIAS Y SISTEMAS

GUATEMALA, ABRIL DE 2026

Documentación Técnica

1. Introducción

El sistema gestiona el canal de distribución On-Line de Industria Típica Guatemalteca, S.A. (ITGSA). Permite llevar control de las facturas generadas por la Tienda Virtual y de los pagos realizados a través de las bancas virtuales de los bancos afiliados.

El sistema está compuesto por dos programas que se comunican mediante HTTP: un backend (API REST) y un frontend (Razor Pages), ambos desarrollados en .NET 9. La información se persiste en archivos XML en el sistema de archivos del servidor.

1.1 Objetivos

- Registrar clientes y bancos de forma incremental mediante archivos config.xml.
- Procesar facturas y pagos mediante archivos transac.xml.
- Aplicar pagos automáticamente a las facturas más antiguas del cliente.
- Gestionar el saldo a favor del cliente cuando un pago excede el total pendiente.
- Exponer una API REST con respuestas en formato XML.
- Proveer una interfaz web simple para la administración del sistema.

1.2 Alcance

El sistema cubre el registro de clientes y bancos, la carga de transacciones, la consulta de estado de cuenta por cliente y el resumen de ingresos por banco. No incluye autenticación de usuarios ni integración directa con sistemas bancarios externos.

2. Arquitectura

El sistema sigue una arquitectura cliente-servidor. El frontend consume los servicios del backend mediante peticiones HTTP. Ambos programas corren en puertos distintos del mismo equipo.

2.1 Componentes

- Frontend (Puerto 5001): Aplicación Razor Pages. Permite cargar archivos XML, consultar datos y visualizar resultados.
- Backend (Puerto 5000): API REST en .NET 9. Procesa los archivos recibidos, aplica la lógica de negocio y persiste los datos en XML.
- Almacenamiento: Cuatro archivos XML en la carpeta Data/Storage/ del backend.

2.2 Stack tecnológico

- Runtime: .NET 9

- Frontend: ASP.NET Core Razor Pages
- Backend: ASP.NET Core Web API
- Persistencia: Archivos XML (System.Xml.Linq)
- Gráficas: Chart.js 4.4.0 (CDN)
- Exportación PDF: window.print() con CSS
-

3. Backend — API REST

El backend expone cinco endpoints que cubren todas las operaciones del sistema. No utiliza base de datos relacional; toda la información se almacena en archivos XML.

3.1 Endpoints

- POST /grabarConfiguracion — Recibe un archivo config.xml mediante multipart/form-data. Registra o actualiza clientes y bancos. Devuelve XML con contadores de creados y actualizados.
- POST /grabarTransaccion — Recibe un archivo transac.xml. Registra facturas y pagos aplicando la lógica de abono. Devuelve XML con contadores de nuevos, duplicados y errores.
- POST /limpiarDatos — Elimina todos los datos del sistema y devuelve confirmación XML.
- GET /devolverEstadoCuenta?nit= — Devuelve el estado de cuenta de un cliente o de todos si no se especifica NIT.
- GET /devolverResumenPagos?mes=&anio= — Devuelve los pagos agrupados por banco para los tres meses anteriores al período indicado.

3.2 Estructura de carpetas

```
ITGSA.API/
├── Controllers/
│   ├── ConfiguracionController.cs
│   ├── TransaccionController.cs
│   └── DatosController.cs
├── Services/
│   ├── ConfiguracionService.cs
│   ├── TransaccionService.cs
│   └── ConsultaService.cs
├── Data/
│   └── DataStore.cs
├── Models/
│   ├── Cliente.cs / Banco.cs / Factura.cs / Pago.cs
│   └── DTOs.cs
└── Helpers/
    └── XmlHelper.cs
```

3.3 Modelos de datos

Cliente

- NIT: identificador alfanumérico único.
- Nombre: razón social del cliente.
- SaldoFavor: monto acumulado a favor del cliente.

Banco

- Codigo: identificador numérico único.
- Nombre: nombre del banco.

Factura

- NumeroFactura: identificador alfanumérico único.
- NITCliente: referencia al cliente.
- Fecha: fecha de emisión en formato dd/MM/yyyy.
- Valor: monto total de la factura.
- SaldoPendiente: monto aún no pagado.
- Pagada: indica si la factura fue liquidada completamente.

Pago

- CodigoBanco: referencia al banco que procesa el pago.
- NombreBanco: nombre del banco al momento del registro.
- Fecha: fecha del pago en formato dd/MM/yyyy.
- NITCliente: referencia al cliente.
- Valor: monto del pago.

3.4 Persistencia XML

El componente DataStore gestiona la lectura y escritura de los cuatro archivos de almacenamiento. Si los archivos no existen al iniciar el sistema, se crean automáticamente vacíos.

- clientes.xml — NIT, nombre y saldo a favor de cada cliente.
- bancos.xml — Código y nombre de cada banco.
- facturas.xml — Todas las facturas con saldo pendiente y estado de pago.
- pagos.xml — Todos los pagos con banco y cliente asociados.

3.5 Configuración

El puerto por defecto del backend es 5000, definido en appsettings.json. El CORS está configurado para aceptar peticiones desde cualquier origen.

```
"Urls": "http://localhost:5000"
```

4. Lógica de Negocio

4.1 Reglas generales

- config.xml es incremental: si el NIT o código de banco ya existe, se actualizan los datos; no se crean duplicados.
- transac.xml es incremental: facturas y pagos son solo de inserción. No se permite actualizar registros existentes.
- Una factura se rechaza si el NITCliente no existe, la fecha es inválida o el valor es menor o igual a cero.
- Un pago se rechaza si el codigoBanco no existe, el NITCliente no existe, la fecha es inválida o el valor es inválido.
- Un pago se considera duplicado si coinciden codigoBanco, NITCliente, fecha y valor con un pago ya registrado.

4.2 Algoritmo de abono de pagos

Cuando se recibe un pago válido, el sistema aplica el siguiente proceso:

- Se obtienen todas las facturas del cliente con saldo pendiente, ordenadas de la más antigua a la más reciente.
- Se itera sobre la lista. Si el monto del pago es mayor o igual al saldo de la factura, la factura se marca como pagada y el proceso continúa con la siguiente.
- Si el monto del pago es menor al saldo de la factura, se abona parcialmente y el proceso termina.
- Si al terminar quedan fondos sin aplicar, se acumulan como saldo a favor del cliente.

4.3 Saldo a favor

Al registrar una nueva factura, el sistema verifica si el cliente tiene saldo a favor. Si lo tiene, abona el mínimo entre el saldo disponible y el valor de la factura antes de guardarla. Si el saldo cubre el total, la factura queda marcada como pagada desde el inicio.

5. Archivos de Entrada y Respuesta

5.1 config.xml

5.2 transac.xml

Archivo incremental de facturas y pagos. No permite actualización de registros ya existentes.

6. Frontend — Razor Pages

El frontend es una aplicación Razor Pages de .NET 10. Consume los endpoints del backend mediante el servicio ApiClient. La interfaz es minimalista con una barra de navegación horizontal y páginas simples.

6.1 Estructura de carpetas

```
ITGSA.Frontend/  
├── Pages/  
│   ├── Shared/_Layout.cshtml  
│   ├── Configuracion/Index  
│   ├── Transacciones/Index  
│   ├── Peticiones/EstadoCuenta  
│   └── Peticiones/Ingresos  
├── Services/  
│   └── ApiClient.cs  
├── Models/  
│   └── ViewModels.cs  
└── wwwroot/  
    ├── css/site.css  
    └── js/site.js
```

6.2 Páginas

- /Configuracion — Carga de config.xml. Muestra contadores de clientes y bancos creados o actualizados.
- /Transacciones — Carga de transac.xml. Muestra seis contadores: nuevas facturas, duplicadas, con error, nuevos pagos, duplicados, con error.
- /Peticiones/EstadoCuenta — Consulta por NIT o todos los clientes. Muestra tabla de transacciones con fecha, descripción, cargo y abono.
- /Peticiones/Ingresos — Selector de mes y año. Muestra gráfica de barras (Chart.js) con pagos por banco en los últimos tres meses, más tabla de detalle.

6.3 Navegación

La navegación es una barra fija en la parte superior con los cuatro enlaces de sección y un botón de Resetear datos. El botón abre un modal de confirmación antes de ejecutar la limpieza.

6.4 Exportación a PDF

Las páginas con datos muestran el botón Imprimir / PDF. Al hacer clic se invoca `window.print()`. El CSS incluye reglas `@media print` que ocultan la navegación y los formularios, dejando solo los datos para imprimir o guardar como PDF desde el navegador.

6.5 Configuración

La URL del backend se define en appsettings.json del frontend mediante la clave ApiUrl.

```
"ApiUrl": "http://localhost:5000"  
"Urls":   "http://localhost:5001"
```

7. Instrucciones de Ejecución

7.1 Requisitos

- .NET 9 SDK instalado.
- Puertos 5000 y 5001 disponibles.
- Conexión a internet para cargar Chart.js desde CDN (solo en la página de Ingresos).

7.2 Pasos

1. Abrir una terminal en la carpeta ITGSA.API y ejecutar:

```
dotnet run
```

2. Abrir una segunda terminal en la carpeta ITGSA.Frontend y ejecutar:

```
dotnet run
```

3. Abrir el navegador en:

```
http://localhost:5001
```

7.3 Orden de operaciones recomendado

- Cargar un archivo config.xml para registrar clientes y bancos.
- Cargar uno o varios archivos transac.xml con facturas y pagos.
- Consultar el estado de cuenta para verificar saldos y transacciones.
- Consultar la gráfica de ingresos seleccionando mes y año.
- Usar Resetear datos para limpiar el sistema y comenzar una nueva prueba.

7.4 Prueba con Postman

El backend puede consumirse directamente desde Postman. Para cargar un archivo usar Body > form-data con la clave archivo de tipo File. Para los endpoints GET agregar los query params directamente en la URL.

Fin.